

Once Generated, Many Times Served: Characterizing and Exploiting Content Recycling in Deep Research Agent Inference

Anonymous submission

Abstract

Deep research agents—multi-agent systems that search, read, and write long, citation-grounded reports—are bottlenecked by LLM inference. We present the first content-level characterization of deep research inference at the LLM-call layer (800 tasks, 15,296 calls). We found a defining property: **content recycling**. Content is generated once, yet later calls serve it again and again. They re-read it during prefill and re-generate it during decode. However, recycling is pervasive but heterogeneous: some calls copy their inputs almost verbatim; others rewrite and condense (copy rates 8–88%, bimodal). **Once generated, many times served**. Existing techniques cannot collect this waste: (1) cross-request KV reuse perturbs downstream decisions in multi-agent pipelines and degrades report quality; (2) uniform speculative decoding wins on copy-heavy calls but loses on rewrite-heavy ones. We exploit recycling on both sides of inference. On the prefill side, we reuse upstream KV caches and preserve downstream decisions via attention-guided selective recomputation. On the decode side, we route speculative decoding by predicted copy rate: the prediction selects the drafter and sets the draft depth. On mainstream deep research benchmarks, our system cuts end-to-end latency by up to 47.5% and TTFT by [TBD]% versus vLLM. Report quality is statistically unchanged.

Introduction

Deep research agents turn a question into an investigation. Given a query, a system such as OpenAI Deep Research (openai-deepresearch 2025) plans sub-questions, searches and reads sources, and writes a long report with citations. Inside, such a system works like a newsroom. A supervisor plans and assigns work. Several researchers each search, read, and write a *summary*. A writer then reads all summaries and produces the final *report*. Each role is a separate LLM call, and one query issues tens of them. The cost is dominated by inference, not retrieval. On the pipeline we study, summary and report generation take about 60% of end-to-end wall clock. Within inference, decode dominates, accounting for about 95% of GPU compute. A serving-layer speedup therefore turns directly into user-visible latency.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

We study this workload at the LLM-call layer. We collect a trace of 800 tasks and 15,296 generation calls from a production-shaped pipeline. For every call, we record its input, its output, and the overlap between them. To our knowledge, this is the first *content-level* characterization of deep research inference. Prior agent traces stop at the systems level. They record token counts and timings but discard the text (tracelab 2026), so copy rates cannot be recovered from them. Prior content measurements sit at other layers: search-query logs (agentic-search-wild 2026), final-report quality (reporteval 2025), or single-turn RAG prompts (byte-exact-dedup 2026). None reach the LLM-call layer of a multi-agent pipeline.

The trace reveals one property that shapes the rest of the paper: **content recycling**. A researcher copies retrieved passages into its summary. The writer copies those summaries into the report. A span of text is thus authored once but used many times. The serving stack does not exploit this. It treats each call as independent and each output token as new. It pays once to generate the span, again to read it in a downstream prefill, and a third time to regenerate it in a downstream decode. *Once generated, many times served*.

Recycling is pervasive, but it is not uniform, and the non-uniformity has two faces. First, the *amount* of copying varies across calls. Report-writing calls copy their input almost verbatim. Summary calls instead rewrite and condense it. Across the trace, per-task copy rates span 8% to 88%, and the distribution is bimodal. No single accelerator fits both ends, so the system must choose per call. Second, when content does recur, it recurs *word for word*. Among summaries of the same task, exact-match overlap averages 22.9%, while a paraphrase-tolerant measure yields only 7.7%. Recurrence is copying, not restatement. Exact-match machinery is therefore enough, and approximate matching is unnecessary.

Two families of technique already target repeated content, and both fail here. The first is cross-request KV reuse. Prefix caching reuses the KV of an identical prompt prefix (radix-attention 2023). But a writer reads its summaries in arbitrary order and subset, so a shared prefix rarely exists; the closest agent-aware system concedes that reordered reuse forces full recomputation (plato 2025). Position-independent reuse removes that constraint by blending precomputed KV into the new prompt (cacheblend 2025; kvcomm 2025; droidspeak 2026). The blended KV is approximate, however. In a multi-

agent pipeline it shifts what a downstream agent attends to. That shift changes the agent’s decisions and degrades report quality. The second family is speculative decoding. Suffix-based speculation drafts the next tokens directly from repeated content (suffixdecoding 2024), and it runs identically on every call. On copy-heavy report calls it wins: 61.2% of drafts are accepted. On rewrite-heavy summary calls the draft rarely matches (28.8% accepted), and the wasted drafting is pure overhead; under concurrency it becomes a net slowdown. Adaptive variants tune the draft length or the drafter (specdecpp 2024; banditspec 2025), but they react to acceptance measured at runtime, not to the copy structure that is knowable before generation.

We exploit recycling on both sides of inference, and we let the copy structure drive both. On the **prefill side**, we reuse the KV that an upstream call already computed while decoding the content. To keep this reuse from perturbing decisions, we recompute selectively. We find the few tokens that downstream attention weights most heavily and recompute only those; the rest are reused unchanged. This preserves the downstream decision while still skipping most of the prefill. On the **decode side**, we predict each call’s copy rate from the model’s hidden state, and we route speculation on that prediction. The prediction selects the drafter — suffix matching for copy-heavy calls, a specialized draft head for rewrite-heavy calls. It also sets the draft depth: deep where copying is high, shallow or off where it is low. The two sides reinforce each other. Our decode-side fix lowers per-step cost but raises TTFT, and the prefill-side reuse pays that TTFT back.

We evaluate on mainstream deep research benchmarks against vLLM. The system cuts end-to-end latency by up to 47.5% and TTFT by [TBD]%. Report quality is statistically unchanged under blind LLM-judge evaluation. This paper makes four contributions:

1. **Characterization.** The first content-level characterization of deep research inference at the LLM-call layer. We identify content recycling and its two axes of heterogeneity, in amount and in form, and we show that copy rate is the signal an accelerator must route on.
2. **Prefill side.** Decision-preserving KV reuse. We reuse upstream KV across a multi-agent pipeline and recompute only the tokens that downstream attention weights most, which avoids the quality loss of blind blending.
3. **Decode side.** Copy-rate-routed speculative decoding. A predictor drives per-call drafter selection and draft depth, so speculation helps copy-heavy calls without taxing rewrite-heavy ones.
4. **System and evaluation.** An integrated system that couples the two sides, evaluated end-to-end on deep research benchmarks, with a quality check and honest negative results that bound where recycling-based acceleration applies.

References

agentic-search-wild. 2026. Agentic Search in the Wild, arXiv:2601.17617 (CTAR). arXiv:2601.17617. PLACEHOLDER generated by md2tex.py – replace with real bibtex.

banditspec. 2025. BanditSpec, arXiv:2505.15141. arXiv:2505.15141. PLACEHOLDER generated by md2tex.py – replace with real bibtex.

byte-exact-dedup. 2026. Byte-Exact Deduplication in RAG, arXiv:2605.09611. arXiv:2605.09611. PLACEHOLDER generated by md2tex.py – replace with real bibtex.

cacheblend. 2025. CacheBlend, EuroSys 2025, arXiv:2405.16444. arXiv:2405.16444. PLACEHOLDER generated by md2tex.py – replace with real bibtex.

droidspeak. 2026. DroidSpeak, NSDI 2026, arXiv:2411.02820. arXiv:2411.02820. PLACEHOLDER generated by md2tex.py – replace with real bibtex.

kvcomm. 2025. KVCOMM, NeurIPS 2025, arXiv:2510.12872. arXiv:2510.12872. PLACEHOLDER generated by md2tex.py – replace with real bibtex.

openai-deepresearch. 2025. OpenAI Deep Research (system card / product page, 2025). PLACEHOLDER generated by md2tex.py – replace with real bibtex.

plato. 2025. Plato, COLM 2025, arXiv:2402.12280. arXiv:2402.12280. PLACEHOLDER generated by md2tex.py – replace with real bibtex.

radixattention. 2023. SGLang/RadixAttention, arXiv:2312.07104. arXiv:2312.07104. PLACEHOLDER generated by md2tex.py – replace with real bibtex.

reporteval. 2025. Understanding DeepResearch via Reports, arXiv:2510.07861. arXiv:2510.07861. PLACEHOLDER generated by md2tex.py – replace with real bibtex.

specdecpp. 2024. SpecDec++, arXiv:2405.19715. arXiv:2405.19715. PLACEHOLDER generated by md2tex.py – replace with real bibtex.

suffixdecoding. 2024. SuffixDecoding, arXiv:2411.04975. arXiv:2411.04975. PLACEHOLDER generated by md2tex.py – replace with real bibtex.

tracelab. 2026. TraceLab, arXiv:2606.30560 (coding-agent trace characterization). arXiv:2606.30560. PLACEHOLDER generated by md2tex.py – replace with real bibtex.